

Plan de Estudio Conceptual

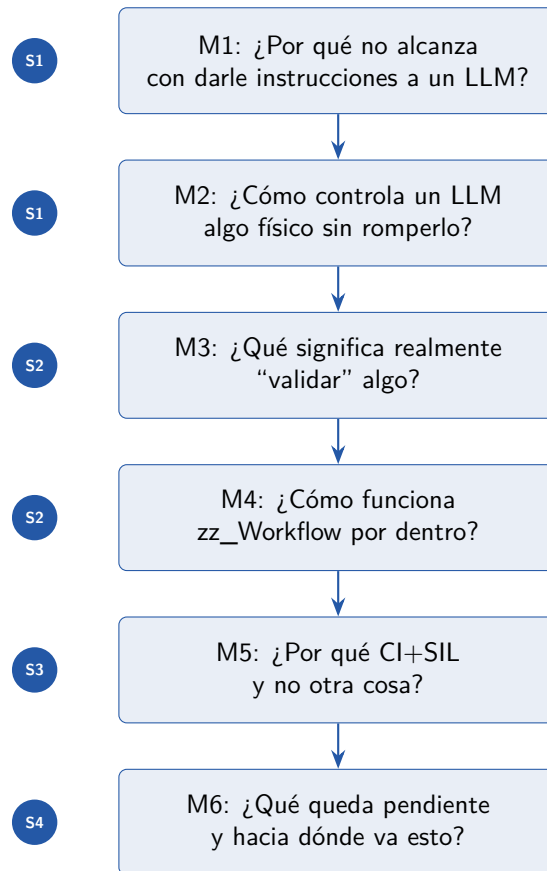
CI+SIL × zz_Workflow

Ruta de aprendizaje para comprender en profundidad los conceptos, principios y decisiones de diseño detrás de la metodología de verificación continua acoplada al motor de prototipado asistido por LLM.

Autor	David Ramírez-Betancourt
Afiliación	Universidad Nacional de Colombia — GCPDS
Fecha	Octubre 2026
Enfoque	Conceptual — sin código ni matemáticas
Tiempo total	~4 semanas (ritmo medio: 6–8 h/semana)
Prerequisitos	Curiosidad y ganas de leer con atención

Cómo usar este plan. Cada módulo tiene: una pregunta central que lo guía, lecturas con secciones específicas, preguntas de reflexión, y un entregable escrito. Los módulos se construyen uno sobre otro. No hay código ni fórmulas: el foco está en entender el **por qué** de cada decisión de diseño.

1. Mapa de la Ruta



2. Módulo 1: ¿Por qué no alcanza con darle instrucciones a un LLM?

Semana 1, primera mitad — 4 h

Si un LLM puede generar código, diseñar arquitecturas y razonar sobre problemas complejos, ¿por qué necesita una estructura rígida alrededor? ¿Qué sale mal cuando lo dejás solo?

2.1 Ideas clave a entender

- **El LLM es probabilístico.** Incluso a temperatura cero, hay una “temperatura de fondo” que hace que dos ejecuciones idénticas puedan dar resultados distintos. La repetibilidad no se puede *asumir*; hay que *medir*.
- **Se degrada en cadenas largas.** Cuando un agente ejecuta muchos pasos secuenciales, la probabilidad de éxito cae rápidamente. Hay un techo práctico de ~ 16 pasos antes de que todos los modelos colapsen. La implicación: los stages tienen que ser cortos y acotados.
- **Los métricos de accuracy incentivan mentir.** Si el benchmark castiga “no sé” tanto como una respuesta incorrecta, el modelo aprende a adivinar con confianza en lugar de abstenerse. Consecuencia: la auto-evaluación del modelo no es confiable.
- **Estructura > instrucciones.** Ponerle más texto al prompt no mejora la confiabilidad tanto como darle interfaces tipadas, loops de verificación y gates explícitos. El contexto (qué ve el

modelo en cada paso) importa más que cómo se lo pedís.

2.2 Lecturas

- L1.** Mittal, “How Fast Do Agents Rot?,” arXiv:2609.01660 — §1–3.
- L2.** Kalai et al., “Evaluating LLMs for accuracy incentivizes hallucinations,” *Nature* 653, 2026 — abstract y §2.
- L3.** Anthropic, “Effective context engineering for AI agents,” 2026 — completo.

2.3 Preguntas de reflexión

- R1.** Si un modelo tiene 97% de éxito por paso, ¿por qué eso no es suficiente para una cadena de 20 pasos? Explicar la intuición sin fórmulas.
- R2.** ¿Qué significa “structure beats instructions”? Dar una analogía de la vida cotidiana (ejemplo: semáforo vs. cartel de “maneje con cuidado”).
- R3.** Si el modelo incentiva adivinar en vez de decir “no sé”, ¿podemos confiar en que él mismo nos diga cuándo se equivocó?

Escrito de 1 página: “Por qué un LLM necesita estructura externa” — explicado como si se lo contaras a un colega que nunca usó un agente de IA.

3. Módulo 2: ¿Cómo controla un LLM algo físico sin romperlo?

Semana 1, segunda mitad — 4 h

Un LLM solo produce texto. ¿Cómo llega ese texto a mover una antena, cambiar una frecuencia o encender un transmisor? ¿Qué pasa si se equivoca?

3.1 Ideas clave a entender

- **Tools como manos del agente.** El LLM no toca el hardware directamente. Llama a *tools* (funciones con parámetros tipados) a través de un protocolo (MCP). El tool traduce la intención del LLM en una acción física.
- **El lazo cerrado.** Agente → Tool → Actuador → Planta física → Sensor → feedback al agente. Sin este feedback, el agente opera a ciegas.
- **Control de dos niveles.** El LLM piensa lento y planifica; un controlador determinístico ejecuta el lazo rápido. El LLM **nunca** está en el fast loop — sería demasiado lento e impredecible.
- **Dos barreras humanas distintas:**
 - **HITL** (Human in the Loop) = barrera de *autorización*. “¿Puedo hacer esto?”
 - **HIL** (Hardware in the Loop) = barrera de *validación*. “¿Esto funciona en el hardware real?”
- **Límites físicos en el esquema.** Si el tool define que la frecuencia máxima es 1766 MHz, una llamada con 2000 MHz se rechaza *antes* de llegar al hardware. Es la primera defensa.

3.2 Lecturas

- L1.** MCP Specification rev. 2025-06-18 — modelcontextprotocol.io — sección “Architecture” y “Tools”.
- L2.** Sadeqi Bajestani et al., “HITL+LLM in smart manufacturing,” *J. Manufacturing Systems* 86, 2026 — §1 y §3.
- L3.** “LLMs Add Safety Risks to Physical AI,” SemiEngineering, Nov. 2025 — completo (incluye el caso del robot de ajedrez que le fracturó un dedo a un niño).

3.3 Preguntas de reflexión

- R1.** ¿Por qué el LLM no puede estar en el fast loop de un controlador? Pensar en términos de velocidad de respuesta y predictibilidad.
- R2.** Explicar la diferencia entre HITL y HIL con un ejemplo concreto del proyecto SDR.
- R3.** Si un agente puede llamar 50 tools distintos, ¿eso lo hace más capaz o más peligroso? ¿Cómo se mitiga?

Diagrama dibujado a mano (o en cualquier herramienta) del lazo cerrado agente–hardware, marcando dónde entra HITL, dónde entra HIL, y dónde están los límites físicos del schema. Añadir 3 oraciones explicando por qué el LLM planifica pero no ejecuta.

4. Módulo 3: ¿Qué significa realmente “validar” algo?

Semana 2, primera mitad — 4 h

Todo el mundo dice “hay que validar”. Pero, ¿validar contra qué? ¿Quién decide si pasó? ¿Es lo mismo verificar que validar?

4.1 Ideas clave a entender

- **Verificar $\neq$ validar.**
 - *Verificación*: “¿Lo construí bien?” — el producto cumple su especificación interna.
 - *Validación*: “¿Construí lo correcto?” — el producto resuelve el problema real del usuario.
- **Nadie valida en un solo paso.** Se valida en capas que van de lo barato a lo caro: primero en software, después en simulación, después en hardware, después con humanos.
- **El oracle no puede ser el generador.** Si el mismo componente que genera la respuesta también la evalúa, la presión de optimización explota las debilidades del evaluador, no mejora la respuesta real.
- **El humano actúa en los extremos.** Al principio (encuadre del problema) y al final (juicio experto). En el medio, los chequeos deben ser lo más automáticos y determinísticos posible.
- **Cinco enfoques conocidos:**
 1. V-model: trazabilidad requisito $\leftrightarrow$ test.

2. CI+SIL: regresión automática en software simulado.
 3. Bancos HIL: hardware real en el lazo.
 4. LLM-as-judge + HITL: el modelo evalúa, el humano arbitra.
 5. MCDA: decisión multi-criterio para elegir entre alternativas.
- **El flujo de 6 fases** organiza estos enfoques en secuencia: Encuadre → Modelo → Software/CI → Banco HIL → Juicio → Decisión MCDA, con gates entre cada fase.

4.2 Lecturas

- L1.** Sheridan Tech, “Design Validation vs Verification,” Mar. 2026 — completo.
- L2.** OPAL-RT, “Guide to HIL Testing,” Sep. 2026 — §1–3 (qué es, cuándo usarlo).
- L3.** “Autonomous Research Agents: Verification Gap,” arXiv:2608.05179 — completo (el 83% publica código, pero solo el 38% da artefactos reproducibles).

4.3 Preguntas de reflexión

- R1.** Explicar con una analogía de construcción de una casa la diferencia entre verificar y validar.
- R2.** ¿Por qué el oracle no puede ser el generador? Imaginar que un estudiante se pone su propia nota — ¿qué incentivos tiene?
- R3.** ¿Por qué validar en capas de barato a caro es más eficiente que ir directo al test más completo?
- R4.** De las 6 fases, ¿cuál es la más barata de ejecutar? ¿Cuál es la más cara? ¿Por qué están en ese orden?

Diagrama del flujo de 6 fases con los gates, incluyendo para cada fase: quién es el oracle y qué pregunta responde. Más un párrafo sobre “verificar vs. validar” en tus propias palabras.

5. Módulo 4: ¿Cómo funciona zz_Workflow por dentro?

Semana 2, segunda mitad — 4 h

zz_Workflow es el motor que produce prototipos verificados. ¿Cómo está organizado internamente? ¿Qué reglas lo gobiernan? ¿Qué produce y qué NO produce?

5.1 Ideas clave a entender

- **4 stages acotados** (Ctx 00–03), cada uno con un contexto específico:
 - Ctx 00: Definir el problema y congelar el alcance.
 - Ctx 01: Fijar requisitos y criterios de aceptación.
 - Ctx 02: Elegir la arquitectura y el plan de implementación.
 - Ctx 03: Contrato de implementación (qué entra, qué sale, qué restricciones tiene).
- **Gates humanas.** Entre cada stage hay un humano que decide si se avanza. El modelo redacta pero **nunca aprueba** su propia salida.

- **La regla de oro:** “La aceptación se decide por criterios pre-declarados aplicados a mediciones registradas, nunca por juicio del modelo.”
- **6 artefactos gobernantes** que definen cómo opera el workflow:
 1. Mapa de arquitectura de prompts.
 2. Especificaciones de RAG (qué documentos alimentan al modelo).
 3. **Suites de evaluación y benchmarking** — el Artefacto 3, que corre chequeos de regresión después de cualquier cambio.
 4. Guardrails (reglas determinísticas que el modelo no puede violar).
 5. Telemetría (registro de qué pasó, qué falló y por qué).
 6. Privacidad y gobernanza.
- **Routing de defectos R0–R3.** Cuando algo falla, no se dice simplemente “falló”. Se clasifica el defecto y se devuelve al stage responsable:
 - R0 → el problema estaba mal definido (volver a Ctx 00).
 - R1 → los requisitos estaban incompletos (volver a Ctx 01).
 - R2 → la arquitectura era inadecuada (volver a Ctx 02).
 - R3 → la implementación tiene un bug (volver a Ctx 03).
- **Salida terminal:** Prototipo Verificado + Evidencia. Es explícitamente *verificación*, no validación. El workflow es honesto sobre lo que entrega.

5.2 Lecturas

- L1.** `context/golden_ctx_CI-SIL+_zz/zz_Workflow_.md` — completo, es el documento fuente.
- L2.** Takahara & Mizoguchi, “Toward Auditable AI Scientists,” arXiv:2607.09195 — §1–2 (por qué importa la auditabilidad).

5.3 Preguntas de reflexión

- R1.** ¿Por qué los stages son “acotados por contexto” en vez de ser etapas libres? ¿Qué problema resuelve acotar lo que el modelo ve en cada momento?
- R2.** Explicar la regla de oro con un ejemplo: si el modelo dice “el detector funciona bien”, ¿por qué eso no cuenta como aceptación?
- R3.** ¿Qué diferencia hay entre decir “el Pd falló” (sin routing) y decir “R2: la arquitectura es inadecuada, volver a Ctx 02”? ¿Por qué el segundo es más útil?
- R4.** El workflow produce un *Prototipo Verificado*. ¿Qué falta para que sea un *Prototipo Validado*? ¿Quién o qué proporciona esa pieza faltante?

Diagrama de zz_Workflow con los 4 stages, las gates humanas, los 6 artefactos, y las rutas R0–R3. Añadir una nota al pie del diagrama: “Produce verificación, no validación — y eso es intencional.”

6. Módulo 5: ¿Por qué CI+SIL y no otra cosa?

Hay 5 metodologías de validación disponibles. ¿Por qué CI+SIL es el acople natural con zz_Workflow? ¿Qué hace que el ajuste sea “HIGH/clean” y no forzado?

6.1 Ideas clave a entender

- **El Artefacto 3 ya ES un harness de regresión.** No hay que inventar nada nuevo. Las suites de evaluación que zz_Workflow ya requiere son exactamente lo que CI necesita ejecutar. El acople es natural, no forzado.
- **El trigger no es solo un commit de código.** En un sistema asistido por LLM, un cambio de modelo, de prompt o de contexto puede alterar el comportamiento sin tocar una sola línea de código. CI+SIL lo reconoce: el trigger es cualquier cambio en modelo, prompt o Ctx.
- **SIL = replay controlado.** Software-in-the-Loop significa que se reproducen señales sintéticas (golden I/Q) sin usar RF real. Es barato, repetible y determinístico.
- **Omitir LLM-as-judge para aceptación es una fortaleza.** Cuando el mismo componente que propone también evalúa, la optimización converge hacia explotar las debilidades del evaluador. La receta: separación de roles + gates determinísticos donde “el rechazo mecánico le gana a la aprobación del LLM, nunca al revés”.
- **La coupling matrix:**

Metodología	Acople	Razón
CI+SIL	HIGH / clean	Artefacto 3 ya es el harness
V&V (V-model)	HIGH-MEDIUM	Compatible pero pesado
LLM-judge + HITL	MEDIUM	Útil para revisión, no para aceptación
Bancos HIL	MEDIUM-LOW	Necesita infraestructura física
MCDA	LOW	Aplica a selección multi-hipótesis

- **Cobertura de las 6 fases:**
 - Fases 1, 3, 5: **cubiertas** (encuadre, software/CI, juicio humano).
 - Fase 2: **parcial** (hay arquitectura y golden I/Q, pero falta un modelo físico predictivo independiente).
 - Fases 4 y 6: **gaps** (HIL real y MCDA quedan downstream).

6.2 Lecturas

- L1.** [context/golden_ctx_CI-SIL+_zz/Acople de zz_Workflow.html](#) — completo.
- L2.** “LLM-as-a-Judge Is Not an Oracle,” arXiv:2609.02246 — completo.
- L3.** [context/golden_ctx_CI-SIL+_zz/CI+SIL × zz_Workflow.html](#) — explorar los 3 tabs del diagrama interactivo.

6.3 Preguntas de reflexión

- R1.** Explicar en tus palabras por qué “el Artefacto 3 ya es un CI harness” es el argumento central del acople.

- R2.** ¿Por qué el trigger de CI debe incluir cambios de prompt y modelo, no solo commits de código? Dar un ejemplo donde un cambio de prompt rompe el comportamiento sin cambiar código.
- R3.** Un colega propone usar LLM-as-judge para decidir si el prototipo pasa. ¿Qué le responderías y por qué? Usar el argumento de arXiv:2609.02246.
- R4.** De los dos gaps (HIL y MCDA), ¿cuál es más urgente de cerrar para el proyecto SDR y por qué?

Ensayo de 2 páginas: “Por qué CI+SIL es el primer tier de integración para zz_Workflow” — incluir la coupling matrix, el argumento del Artefacto 3, la razón para omitir LLM-as-judge, y los gaps pendientes.

7. Módulo 6: ¿Qué queda pendiente y hacia dónde va esto?

Semana 4 — 6 h

CI+SIL cubre la verificación continua. Pero “verificado” no es “validado”. ¿Qué falta? ¿Cuál es el camino desde un prototipo verificado hasta uno validado?

7.1 Ideas clave a entender

- **“Verificado en simulación” ≠ “validado”.** El claim debe nombrar el ambiente en el que se produjo la evidencia. Un prototipo que pasa todos los tests con señales sintéticas NO está validado en hardware real.
- **Gap 1 — Banco HIL.** Para ir de verificación a validación se necesita un banco de hardware in the loop: SDR real, señales RF reales (o capturadas), y mediciones físicas. Esto cierra la fase 4.
- **Gap 2 — MCDA.** Cuando hay múltiples hipótesis de diseño (distintas arquitecturas de detector, por ejemplo), se necesita un método de decisión multi-criterio para elegir entre ellas. Esto cierra la fase 6. No aplica mientras haya un solo prototipo.
- **Gap 2b — Modelo físico independiente.** La fase 2 es parcial porque no hay un modelo predictivo que actúe como oracle independiente del código. Un modelo analítico de propagación FM podría cerrar esto.
- **La honestidad del nombre.** Llamar al output “Prototipo Verificado” y no “Prototipo Validado” no es modestia: es precisión. Mantener esta distinción hace que los claims sean tractables y las próximas etapas estén claras.

7.2 Lecturas

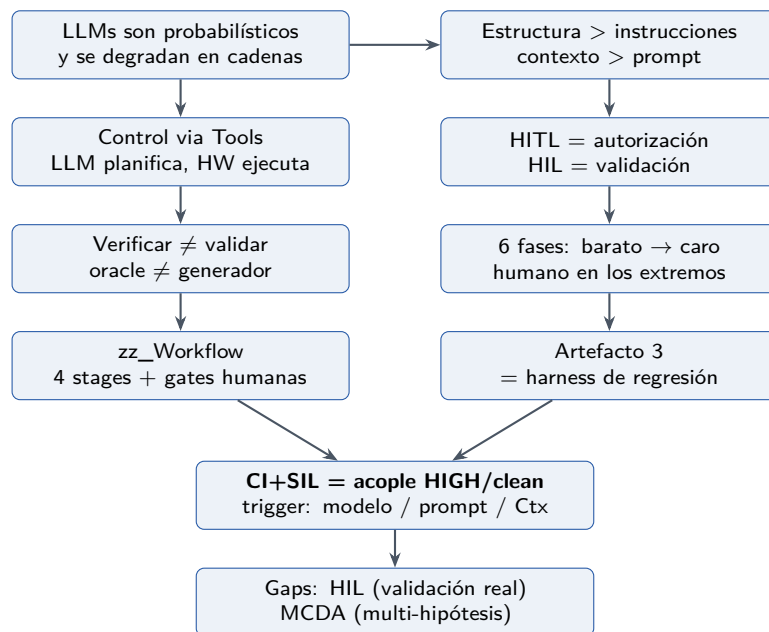
- L1.** Heinrichs et al., “From Prompt to Prototype: LLM-Driven RF Workflow,” arXiv:2608.31006 — §4–5 (gap entre simulación y fabricación/medición).
- L2.** OPAL-RT, “Guide to HIL Testing,” Sep. 2026 — §4–6 (cómo montar un banco HIL).
- L3.** *Sustainable Futures*, “MCDA reliability,” Jun. 2026 — §1–3 (cuándo usar y cuándo no usar MCDA).

7.3 Preguntas de reflexión

- R1.** Dibujar la secuencia completa: $zz_Workflow \rightarrow CI+SIL \rightarrow \text{Banco HIL} \rightarrow \text{Validación}$. ¿Dónde cambia el tipo de evidencia de “sintética” a “real”?
- R2.** ¿Por qué es importante *nombrar* la diferencia entre verificado y validado, en vez de usar los términos de forma intercambiable?
- R3.** Si tu proyecto tiene un solo diseño de detector, ¿necesitás MCDA? ¿Cuándo empezaría a ser relevante?
- R4.** Imaginar que montás el banco HIL y el detector falla con RF real pero pasaba en simulación. ¿A qué ruta R lo devolverías? ¿Por qué?

Mapa conceptual completo de la metodología, desde los fundamentos de LLM hasta los gaps pendientes. Debe responder las 4 preguntas del criterio de completitud (ver abajo). Formato libre: mapa mental, diagrama, o texto estructurado.

8. Criterio de Completitud



Sabés que dominaste la metodología si podés explicar a otro ingeniero:

- 1. Por qué un LLM no puede auto-verificarse (el oracle no es el generador).*
- 2. Qué produce zz_Workflow (prototipo verificado + evidencia) y qué NO produce (validación).*
- 3. Por qué el Artefacto 3 ya es un CI harness (el acople no se inventa, se descubre).*
- 4. Qué falta para ir de “verificado” a “validado” (HIL, modelo físico independiente).*